

INTRODUCCIÓN AL DESARROLLO WEB

Introducción



Objetivos

- Conocer la arquitectura cliente-servidor y la infraestructura web moderna
- Presentar los modelos de programación en entornos cliente y servidor.
- Comprender el protocolo HTTP/HTTPS, su evolución y los mecanismos de comunicación como REST, WebSockets, GraphQL
- Identificar los lenguajes más importantes para el desarrollo de aplicaciones web.
- Conocer los principales lenguajes y frameworks para el desarrollo en el lado del servidor.
- Entender cómo se integran los lenguajes de programación con los lenguajes de marcas y los servidores.
- Aplicar buenas prácticas de calidad de software mediante Código Limpio, principios SOLID y Patrones de diseño.



Índice

1. [Páginas web estáticas y dinámicas](#)
2. [Servidor web y servidor de aplicaciones web](#)
3. [El protocolo HTTP, APIs y autenticación](#)
4. [Arquitecturas y plataformas de desarrollo](#)
5. [Modelos de programación en entornos cliente-servidor](#)
6. [Desarrollo en capas](#)

7. Single Page Application(SPA) y modelos de renderizado moderno
8. Lenguajes de programación en entorno servidor y Frameworks
9. Integración con los lenguajes de marcas
10. Código limpio y principios SOLID
11. Patrones de diseño y MVC
12. Infraestructura y despliegue en aplicaciones web
13. Mapa conceptual

1. Páginas web estáticas y dinámicas

El lenguaje básico para la web es el HTML. Una página que esté escrita usando solo HTML será estática, es decir, mostrará siempre el mismo contenido.

Cuando se utiliza un lenguaje de programación en el servidor se pueden generar páginas en función de la solicitud del cliente.

Por ejemplo, muchas páginas muestran anuncios diferentes según dónde esté el usuario. Es decir, no se ofrece el mismo contenido a todos los usuarios, sino que parte del contenido se genera dinámicamente. En este caso se habla de páginas web dinámicas.

Páginas web estáticas

En este tipo de páginas la información mostrada es siempre la misma y no permiten la interacción con el usuario para modificar el contenido del documento. Son muy útiles para la creación de páginas exclusivamente informativas, desplegar avisos de privacidad o landing pages de presentación corporativa.

La única forma en que pueden cambiar es si un programador la modifica y actualiza su contenido. Su actualización depende de una modificación en la base de datos que alimenta al sitio y no de la interacción con el visitante.

A pesar de estas restricciones existen algunas funcionalidades que permiten la interacción con el usuario en páginas web estáticas. Por ejemplo, cuando un visitante desea cambiar el idioma de un sitio, lo que hace el sistema es redirigirlo a otra página web que tiene el mismo contenido en el lenguaje seleccionado. En términos formales el contenido no cambia y será el mismo en cada visita a la URL de la página, pero sí existe la posibilidad de crear una idea de interactividad entre distintas páginas.

Cuando una página web se descarga a tu ordenador, su contenido define qué se debe mostrar en pantalla. Este contenido está programado en un lenguaje de marcado, formado por etiquetas.

Su contenido está escrito directamente en lenguaje de marcado HTML para la estructura y hojas de estilo CSS para el diseño visual.

El esquema de funcionamiento es el siguiente:

1. El navegador del cliente solicita un archivo .html o .css al servidor web mediante una petición HTTP.
2. El servidor busca dicho archivo en el sistema de almacenamiento.
3. Si el archivo existe, el servidor lo envía directamente al cliente sin realizar ningún procesamiento intermedio.
4. El navegador del cliente interpreta el código y renderiza la interfaz.

Para visualizar una página estática sólo es preciso un navegador web. No hace falta ningún otro software.

Páginas web dinámicas

Una página web dinámica permite la interacción con el usuario para crear experiencias personalizadas y únicas. Esto se puede llevar a cabo a través de formularios, herramientas interactivas o recursos multimedia disponibles para la navegación.

Los contenidos de una página web dinámica varían con cada carga para ofrecer actualizaciones en tiempo real y ajustarse a las necesidades del visitante. El contenido cambia en función de diversas variables, como puede ser el navegador que estás usando, el usuario con el que te has identificado, o las acciones que has efectuado con anterioridad.

Los sitios web dinámicos basan su comportamiento y funcionalidad en dos tipos de programación:

- Front-end (del lado del cliente): Código JavaScript/TypeScript que se ejecuta directamente en el navegador del usuario.
- Back-end (del lado del servidor): Instrucciones ejecutadas en el servidor utilizando lenguajes como Node.js, Python, PHP, Java o C#.

La combinación de ambas capas y el uso de tecnologías como AJAX (Asynchronous JavaScript and XML) o la API fetch permiten solicitar datos al servidor y actualizar zonas específicas del documento sin necesidad de recargar la página completa..

Una vez ejecutadas las instrucciones en el servidor, un nuevo HTTP response se envía al navegador del usuario para mostrarle lo que ha solicitado. El resultado final es el mismo que en un sitio web estático, una página HTML que el usuario ve desde el navegador.

Un ejemplo de página web dinámica: [LinkedIn.com](https://www.linkedin.com/) <https://www.linkedin.com/>



Actividad: Ventajas y desventajas

Busca información acerca de las ventajas y desventajas de una página web estática y de una página web dinámica.

2. Servidor Web y servidor de aplicaciones

Un **Servidor Web** es una combinación de hardware y software que utiliza los protocolos HTTP y HTTPS para servir archivos a los usuarios a través de la red.

En términos sencillos, un servidor web es un ordenador que almacena, procesa y entrega archivos de sitios web a los usuarios desde un navegador.

Función: Recibe las peticiones HTTP, gestiona las conexiones cifradas (TLS/SSL) y sirve archivos estáticos (HTML, CSS, imágenes) o redirige el tráfico a un servidor de aplicaciones



Ejemplos de servidores web: Apache, Nginx, Microsoft IIS

Un **Servidor Aplicaciones** es un entorno de software que añade una capa de lógica de negocio o programación. Se encarga de ejecutar el código backend (Node.js, JVM, .NET) para procesar datos, interactuar con bases de datos y generar respuestas dinámicas (normalmente en formato JSON o HTML).

2. Aplicaciones Web

Una aplicación web es una aplicación informática a la que se accede mediante una interfaz web utilizando un navegador.

Las aplicaciones web emplean **páginas web dinámicas** para crear aplicaciones que **se ejecuten en un servidor web** y se muestren en un navegador.

Hoy en día existen aplicaciones web para multitud de tareas como procesadores de texto, gestión de tareas, edición y almacenamiento de imágenes, etc. Estas aplicaciones tienen ciertas ventajas e inconvenientes si las comparas con las aplicaciones tradicionales que se ejecutan sobre el sistema operativo de la propia máquina.

Ventajas de las aplicaciones web:

- No es necesario instalarlas en los equipos en que se vayan a utilizar. Se instalan y se ejecutan en el servidor, y esto es suficiente para que se puedan utilizar de forma simultánea desde muchos equipos.
- Como solo se encuentran instaladas en un equipo, es muy sencillo gestionarlas (hacer copias de seguridad de sus datos, corregir errores, actualizarlas).
- Se pueden utilizar en todos aquellos sistemas que dispongan de un navegador web, independientemente de sus características (no es necesario un equipo potente) o de su sistema operativo.
- Se pueden utilizar desde cualquier lugar en el que dispongamos de conexión con el servidor. En muchos casos esto hace posible que se pueda acceder a las aplicaciones desde sistemas no convencionales, como por ejemplo teléfonos móviles.

Inconvenientes de las aplicaciones web:

- La interfaz de usuario de las aplicaciones web es la página que se muestra en el navegador. Esto restringe las características de la interfaz a aquellas de una página web.
- Dependemos de una conexión con el servidor para poder utilizarlas. Si nos falla la conexión, no podremos acceder a la aplicación web.

- La información que se muestra en el navegador debe transmitirse desde el servidor.

Hoy en día muchas aplicaciones web utilizan las ventajas que les ofrece la generación de páginas dinámicas. La gran mayoría de su contenido está almacenado en una base de datos. Aplicaciones como Drupal, Joomla!, etc. ofrecen dos partes bien diferenciadas:

- Una parte externa o front-end, que es el conjunto de páginas que ven la gran mayoría de usuarios que las usan (usuarios externos).
- Una parte interna o back-end, que es otro conjunto de páginas dinámicas que utilizan las personas que producen el contenido y las que administran la aplicación web (usuarios internos) para crear contenido, organizarlo, decidir la apariencia externa, etc.

3. El protocolo HTTP, APIs y Autenticación

HTTP (Hypertext Transfer Protocol) es el protocolo empleado para la transferencia de los recursos en una web.

- **Sin estado** (Stateless): Cada petición entre cliente y servidor es independiente y no guarda memoria de transacciones pasadas por sí sola.
- **Orientado a transacciones:** Sigue el modelo petición-respuesta (request-response) entre un cliente y un servidor.
- **Identificación de Recursos:** Cada recurso en la web se identifica mediante un identificador o localizador único llamado URL (Uniform Resource Locator).

Importancia:

- **Arquitecturas Distribuidas:** HTTP es fundamental en arquitecturas distribuidas como microservicios, donde diferentes servicios se comunican entre sí.
- **Creación de API:** Es la base para la creación de APIs REST (Representational State Transfer), que es una forma de diseñar sistemas de software escalables y fáciles de mantener.

URL

Las URL indican cómo localizar en Internet un determinado recurso.

Su formato es: **protocolo://maquina:puerto/camino/fichero**

- El protocolo es HTTP o HTTPS, aunque pueden emplearse otros protocolos como FTP.
- Máquina es el nombre o la IP del servidor al cual nos queremos conectar
- Puerto: Puerto de red del servicio.
 - HTTP: Usa por defecto el puerto 80.
 - HTTPS: Usa por defecto el puerto 443 (conexión cifrada con TLS/SSL).
- Camino es la ruta en el sistema de archivos de la máquina remota donde se encuentra el recurso al cual queremos acceder. Dicha ruta es relativa al directorio raíz de la web.
- Fichero es el recurso concreto al que queremos acceder dentro de un directorio de la máquina.

Evolución del protocolo HTTP

- **HTTP/1.1:** Basado en texto plano, soporta conexiones persistentes, pero sufre del problema de bloqueo de cabeza de línea (Head-of-Line Blocking) a nivel de aplicación.
- **HTTP/2:** Protocolo binario que introduce multiplexación (envío de múltiples peticiones y respuestas en paralelo sobre una misma conexión TCP), compresión de cabeceras (HPACK) y Server Push.
- **HTTP/3:** Funciona sobre QUIC (un protocolo de transporte basado en UDP en lugar de TCP). Elimina el bloqueo a nivel de transporte, reduce drásticamente el tiempo de establecimiento de conexión y mejora el rendimiento en redes inestables o móviles.

Mensajes HTTP

Los **mensajes HTTP** <<https://developer.mozilla.org/es/docs/Web/HTTP/Messages>> , son los medios por los cuales se intercambian datos entre servidores y clientes.

Constan de una línea inicial, un conjunto de cabeceras (headers), una línea en blanco y un cuerpo opcional (body).

Hay dos tipos de mensajes:

- **peticiones**

<https://developer.mozilla.org/es/docs/Web/HTTP/Messages#peticiones_http> , enviadas por el cliente al servidor, para pedir el inicio de una acción

Tienen la siguiente estructura:

```
Método SP URL SP Versión Http retorno_de_carro
(nombre-cabecera: valor-cabecera (, valor-cabecera)*CRLF)*
Cuerpo del mensaje
```

nota:

* CRLF es retorno de carro

* SP es espacio

* El cuerpo suele ir vacío pero no cuando enviamos formularios, subimos ficheros, ...

Ejemplo:

```
GET /index.html HTTP/1.1
Host: www.sitioweb.com:8080
User-Agent: Mozilla/5.0 (Windows;en-GB; rv:1.8.0.11)
Gecko/20070312
[Línea en blanco]
```

HTTP define un conjunto de [métodos de petición](#)

<<https://developer.mozilla.org/es/docs/Web/HTTP/Methods>> para indicar la acción que se desea realizar para un recurso determinado. En el ejemplo anterior se ha usado GET pero a lo largo del curso usaremos POST, PUT, etc.

- [respuestas del servidor.](#)

<https://developer.mozilla.org/es/docs/Web/HTTP/Messages#respuestas_http>

Tienen la siguiente estructura:

```
Versión-http SP código-estado SP frase-explicación
retorno_de_carro
(nombre-cabecera: valor-cabecera ("," valor-cabecera)*
CRLF)*
Cuerpo del mensaje
```

Ejemplo:

```
HTTP/1.1 200 OK
Content-Type: text/xml; charset=utf-8
Content-Length: 673
<html>
<head> <title> Título de nuestra web </title>
</head>
<body>
¡Hola mundo!
</body>
</html>
```

Las peticiones y respuestas HTTP, comparten una estructura similar, compuesta de:

- * Una línea de inicio describiendo la petición a ser implementada, o su estado, sea de éxito o fracaso.
- * Un grupo opcional de cabeceras HTTP, indicando la petición o describiendo el cuerpo que se incluye en el mensaje.
- * Una línea vacía indicando toda la meta-información ha sido enviada.

* Un campo de cuerpo de mensaje opcional que lleva los datos asociados con la petición (como contenido de un formulario HTML), o los archivos o documentos asociados a una respuesta (como una página HTML, o un archivo de audio, vídeo ,etc) . La presencia del cuerpo y su tamaño es indicada en la línea de inicio y las cabeceras HTTP.

La línea de inicio y las cabeceras HTTP, del mensaje, son conocidas como la cabeza de la petición, mientras que su contenido en datos se conoce como el cuerpo del mensaje.

Las **cabeceras** <https://en.wikipedia.org/wiki/List_of_HTTP_header_fields> aportan gran información. En el primer ejemplo podemos ver el host con el puerto y la identificación del navegador cliente.

Verbos HTTP

Si realizamos CRUD, debemos utilizar los verbos HTTP de forma adecuada:

- GET: Solicita un recurso al servidor (no debe alterar el estado).
- POST: Enviar datos al servidor para crear un nuevo recurso.
- PUT: Actualizar o reemplazar datos existentes en el servidor.
- DELETE: Eliminar datos del servidor.
- PATCH: Actualizar parcialmente un recurso en el servidor.

Códigos de estado

Un código de estado HTTP es un mensaje que el servidor de un sitio envía al navegador para indicar si una solicitud puede ser cumplida o no. Los códigos de estado son marcados por la W3C. Estos son embebidos en el encabezado HTTP de una página para decirle al navegador el resultado de su solicitud.

Ejemplo: **HTTP/1.1 200 OK**

Podemos consultar una lista completa de los posibles estados en [Wikipedia](https://es.wikipedia.org/wiki/Anexo:C%C3%B3digos_de_estado_HTTP) <https://es.wikipedia.org/wiki/Anexo:C%C3%B3digos_de_estado_HTTP> y en [Mozilla Developer](https://developer.mozilla.org/es/docs/Web/HTTP/Status) <<https://developer.mozilla.org/es/docs/Web/HTTP/Status>> .

Códigos de Respuesta más usuales HTTP:

- Códigos 1xx : Mensaje
 - 100-111 Conexión rechazada
- Códigos 2xx: Operación realizada con éxito
 - 200 OK
 - 201 Created: La solicitud ha tenido éxito y se ha creado un nuevo recurso.
 - 204 No Content: La solicitud ha tenido éxito, pero no hay contenido para enviar.
 - 205 Contenido para recargar
 - 206 Contenido parcial
- Códigos 3xx: Redirección
 - 301 Mudado permanentemente
 - 302 Encontrado
 - 304 No modificado
 - 305 Utilice un proxy
 - 307 Redirección temporal
- Códigos 4xx: Error por parte del cliente
 - 400 Solicitud incorrecta
 - 401 Unauthorized: Autenticación necesaria.
 - 402 Pago requerido

- 403 Forbidden: El servidor comprendió la solicitud, pero se niega a cumplirla.
- 404 Not Found: El servidor no puede encontrar el recurso solicitado.
- Códigos 5xx: Error del servidor
 - 500 Internal Server Error: El servidor encuentra una situación que no sabe cómo manejar.
 - 503 Servicio no disponible
 - 504 Tiempo de espera de la pasarela agotado
 - 505 Versión de HTTP no soportada

APIs Web y Estilos de Comunicación

- **API REST**(Representational State Transfer): Estilo arquitectónico que utiliza las propiedades nativas de HTTP (verbos, códigos de estado) para manipular recursos representados habitualmente en formato JSON.
- **GraphQL**: Alternativa a REST creada por Meta que permite al cliente definir en la petición la estructura exacta de los datos que necesita, evitando traer datos innecesarios (over-fetching) o hacer múltiples peticiones .
- **gRPC**: Framework RPC de ultra-alto rendimiento desarrollado por Google, basado en HTTP/2 y Protocol Buffers, ideal para comunicación entre microservicios.
- **WebSockets y SSE**: Permiten comunicación bidireccional en tiempo real (WebSockets) o emisión unidireccional continua del servidor al cliente (Server-Sent Events).

Autenticación y Seguridad Web

- **JWT (JSON Web Token):** Estándar (RFC 7519) para transmitir identidades y permisos de forma segura y sin estado (stateless) entre cliente y servidor. Se compone de tres partes codificadas en Base64Url: Header.Payload.Signature.
- **OAuth 2.0 / OpenID Connect:** Estándares de la industria para autorización delegada (OAuth 2.0) e identificación de usuarios (OpenID Connect).
- **CORS y CSP:**
 - **CORS** (Cross-Origin Resource Sharing): Mecanismo de seguridad de los navegadores para restringir peticiones HTTP a dominios distintos al de origen.
 - **CSP** (Content Security Policy): Cabecera de seguridad que previene ataques de inyección de código como XSS (Cross-Site Scripting).

4. Arquitecturas y plataformas de desarrollo

Una arquitectura de software se refiere a la estructura organizativa fundamental de un sistema de software. Define cómo se dividen, combinan y coordinan los componentes del software para lograr los objetivos del sistema.

Entre las principales arquitecturas encontramos:

- **Arquitectura monolítica:** Todos los componentes de una aplicación web se agrupan en un solo bloque. La lógica de negocio, la interfaz de usuario y la capa de acceso a datos se encuentran dentro de la misma aplicación. Es fácil de desarrollar y desplegar, pero puede volverse complejo y difícil de mantener a medida que la aplicación crece. Características:
 - Todos los componentes se ejecutan en el mismo proceso y comparten recursos.
 - La escalabilidad puede ser un desafío, ya que la aplicación se ejecuta en una sola instancia.
 - Los cambios en una parte de la aplicación pueden afectar a otras partes.
- **Arquitectura de capas:** Esta arquitectura divide la aplicación en diferentes capas lógicas, donde cada capa tiene una responsabilidad específica. Las capas típicas incluyen la capa de presentación, la capa de lógica de negocio y la capa de acceso a datos. Cada capa se comunica con la capa adyacente a través de interfaces bien definidas. Características:
 - Mejora la modularidad y la reutilización del código.
 - Permite cambios en una capa sin afectar a las demás.
 - Facilita la escalabilidad y el mantenimiento del sistema.
- **Arquitectura de servicios web:** Esta arquitectura se basa en la comunicación entre diferentes servicios a través de protocolos web estándar, como HTTP. Cada servicio es una unidad independiente que se puede desarrollar, desplegar y escalar de forma independiente. Los servicios se comunican entre sí para cumplir con los requisitos de la aplicación. Características:
 - Favorece la modularidad y la independencia de los servicios.
 - Permite la integración de diferentes tecnologías y lenguajes de programación.
 - Facilita la escalabilidad horizontal, ya que los servicios se pueden replicar y distribuir en múltiples servidores.

- **Arquitectura basada en microservicios:** Es una evolución de la arquitectura de servicios web, donde los servicios se dividen en componentes aún más pequeños y autónomos llamados microservicios. Cada microservicio se enfoca en una tarea específica y se comunica con otros microservicios a través de protocolos ligeros.

Características:

- Cada microservicio se puede desarrollar, desplegar y escalar de forma independiente.
- Mejora la flexibilidad y la agilidad del desarrollo.
- Permite la adopción de diferentes tecnologías y enfoques dentro de cada microservicio.

Antes de comenzar a programar una aplicación web tendrás que elegir la arquitectura que vas a utilizar, así como las plataformas y tecnologías.

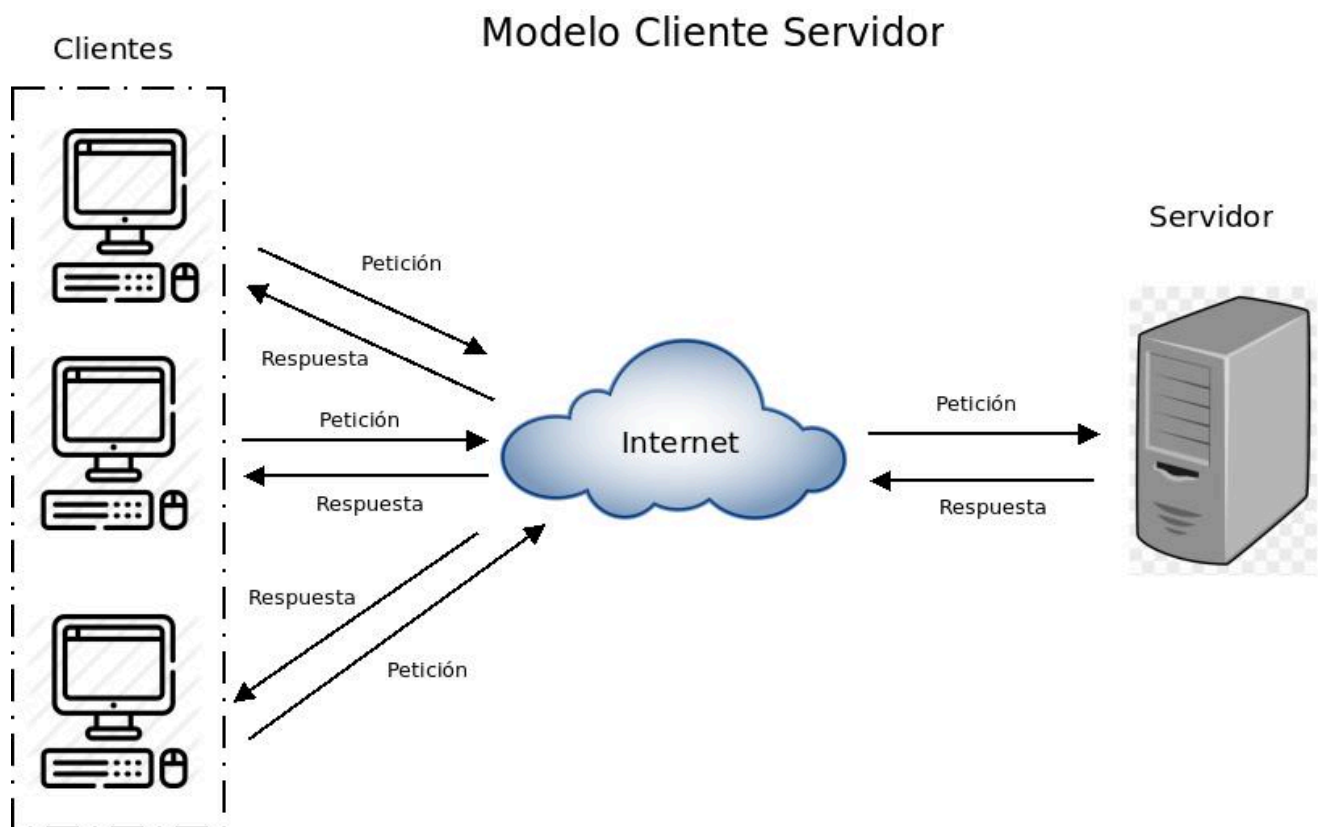
Principales plataformas y Ecosistemas:

- **Ecosistema JavaScript / TypeScript** (Node.js, Deno, Bun): Permite utilizar JavaScript/TypeScript tanto en el cliente como en el servidor. TypeScript se ha convertido en el estándar por defecto por su soporte de tipado estático y escalabilidad.
- **.NET (.NET Core / ASP.NET Core):** Plataforma de desarrollo creada por Microsoft. Es completamente de código abierto (open source), multiplataforma (ejecutable en Linux, macOS y Windows) y de altísimo rendimiento utilizando el lenguaje C#.
- **Jakarta EE** (anteriormente Java EE): Plataforma empresarial en Java basada en especificaciones estandarizadas (JPA, Jakarta Servlets). En la actualidad, el framework Spring Boot domina el desarrollo en este ecosistema.
- **Pila LAMP / WAMP (PHP):** Entorno basado en Linux/Windows, Apache, MySQL y PHP. Sigue siendo una opción popular con frameworks modernos como Laravel o Symfony.
- **Python y Go / Rust:** Python destaca en desarrollo rápido y APIs dinámicas (FastAPI, Django). Go (Golang) y Rust se han consolidado en el desarrollo de microservicios concurrentes y sistemas de alto rendimiento.

5. Modelos de programación en entornos cliente-servidor

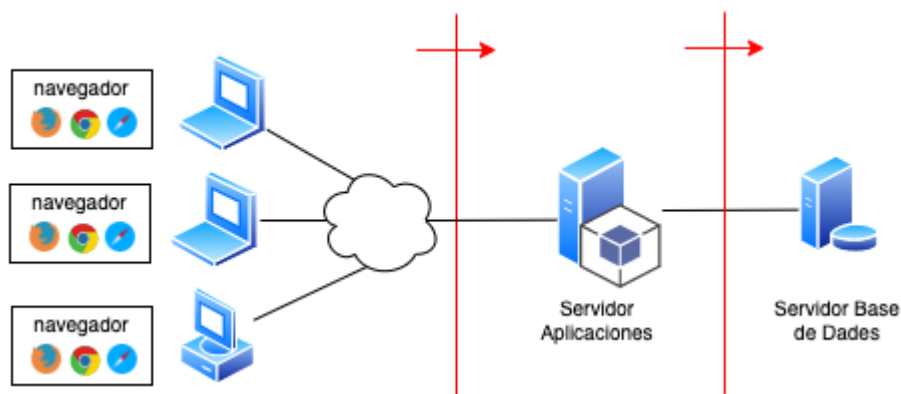
Las aplicaciones web se basan, desde un punto de vista centrado en el hardware, en el modelo cliente-servidor.

- Cliente: Inicia la comunicación enviando peticiones HTTP (generalmente navegadores o aplicaciones móviles).
- Servidor: Escucha peticiones, procesa la lógica de negocio necesaria y devuelve un mensaje de respuesta



Cliente y servidor se comunican mediante un protocolo que define el formato de solicitudes y respuestas y, en general, se ejecutan en máquinas diferentes conectadas por red, aunque también pueden ejecutarse en el mismo ordenador.

En el caso de los servidores web los clientes son, la mayoría de las veces, agentes de usuario (p.e. los navegadores). Los clientes realizan peticiones de páginas al servidor y éste responde con la página web solicitada o con un mensaje de error; si no la encuentra o el acceso no está autorizado.



La comunicación entre clientes y servidores web se realiza mediante protocolo HTTP o su versión segura, HTTPS, que tienen reservados los puertos 80 y 553 respectivamente en la lista de puertos bien conocidos. Las características del modelo cliente-servidor y del protocolo HTTP condicionan el desarrollo de aplicaciones web.

Al programar una aplicación, se utiliza un lenguaje de programación y ciertas herramientas como un entorno de desarrollo o librerías de código. Además, una vez acabado su desarrollo necesitará ciertos componentes para su ejecución, como por ejemplo una máquina virtual de Java.

Los **componentes** principales con los que debes contar para ejecutar aplicaciones web en un servidor son los siguientes:

- Un **servidor web** para recibir las peticiones de los clientes web y enviarles la página que solicitan. Los clientes web suelen ser los navegadores y la página solicitada sólo se envía una vez generada, puesto que hablamos de páginas web dinámicas. El servidor web debe conocer el procedimiento a seguir para generar la página web: qué módulo se encargará de la ejecución del código y cómo se debe comunicar con él.
- El **módulo encargado de ejecutar el código** o programa y generar la página web resultante. Este módulo debe integrarse de alguna forma con el servidor web, y dependerá del lenguaje y tecnología que utilicemos para programar la aplicación web.
- Una **aplicación de base de datos**, que normalmente también será un servidor. Este módulo no es estrictamente necesario, pero en la práctica se utiliza en todas las aplicaciones web que utilizan grandes cantidades de datos para almacenarlos.
- El **lenguaje de programación** que utilizarás para desarrollar las aplicaciones.

6. Desarrollo en capas

Además de elegir los componentes a utilizar, también es importante decidir cómo vas a **organizar el código** de la aplicación. Muchas de las arquitecturas que se usan en la programación de aplicaciones web te ayudan a estructurar el código de las aplicaciones en capas o niveles.

El motivo de dividir en capas el diseño de una aplicación es que se puedan separar las funciones lógicas de la misma, de tal forma que sea posible ejecutar cada una en un servidor distinto (en caso de que sea necesario). [1]

Cada capa puede ocuparse de una o varias de las funciones.

Por ejemplo, en la **arquitectura a tres capas** (ampliación del modelo cliente-servidor) la lógica de la aplicación se separa en:

- a) **Capa de presentación** o de cliente. Es la parte visible de la aplicación. Muestra información y permite interactuar con el sistema al usuario. Aquí se programará todo lo relacionado con la interfaz de usuario.
- b) **Capa de negocio**. En esta capa intermedia se programa la funcionalidad de la aplicación. Se comunica con las otras dos capas. Siempre se ejecuta en el lado del servidor. Esta capa, tras realizar todos los cálculos y/o operaciones sobre los datos, genera el código HTML que será presentado al usuario en la capa siguiente. Los componentes de esta capa pueden ser desde simples archivos HTML, Servlets de Java, ficheros PHP, Python, etc
- c) **Capa de datos**. Para gestionar la base de datos, es decir se encarga de almacenar la información de la aplicación en una base de datos y recuperarla cuando sea necesario. Generalmente es un gestor o gestores de bases de datos (relacionales o no relacionales) pero pueden ser ficheros de texto plano, ficheros XML, JSON, etc.

La capa de presentación no se comunica directamente con la de datos, sino que se hace a través de la capa de negocio. La capa de negocio recibe las acciones del

usuario de la capa de presentación y, si es necesario, se comunica con la capa de datos para consultar o modificar la base de datos. De la misma manera, pasa la salida de la capa de datos a la de presentación.



Figura 3. Modelo a 3 capas.

En una aplicación web, la capa de presentación se ejecuta en un navegador, en el ordenador del usuario; la capa de negocio, en el servidor web, y la de datos, en el servidor de bases de datos. Estas dos pueden estar en equipos diferentes o en el mismo.

El objetivo es desacoplar la lógica de negocio de la de presentación para conseguir código más reutilizable. Además, mediante la separación entre componentes se obtienen aplicaciones más fáciles de mantener y ampliar.

Por ejemplo, al hacer login en una aplicación web:

- Desde el cliente (navegador) se envía un formulario. Es la capa de presentación.
- En el servidor, capa de negocio, se consulta con la base datos (capa de datos) si los datos son correctos.
- En función de la respuesta, se pasan las instrucciones correspondientes a la capa de presentación (se permite el acceso o se muestra un mensaje de error).

[1] En una aplicación puedes distinguir, de forma general, funciones de presentación (se encarga de dar formato a los datos para presentárselo al usuario final), lógica (utiliza los datos para ejecutar un proceso y obtener un resultado), persistencia (que mantiene los datos almacenados de forma organizada) y acceso (que obtiene e introduce datos en el espacio de almacenamiento).

7. SPA y modelos de renderizado

Una **Single-Page Application** (SPA) es una arquitectura en la que la aplicación web carga un único documento HTML base y actualiza dinámicamente el contenido mediante llamadas a APIs (AJAX/Fetch) a medida que el usuario interactúa.

La arquitectura de una SPA se sustenta en la **carga de una sola página** de forma inicial, a la que se van agregando los contenidos necesarios de forma dinámica al cambiar de vista o interactuar. Para esas “cargas”, se va **solicitando el contenido al servidor** utilizando Ajax o WebSocket.

Como se ejecutan en el navegador, los sitios single page utilizan el lenguaje JavaScript para su frontend, complementado con tecnologías HTML y CSS. Lo más habitual es utilizar **librerías y frameworks**, entre los que debemos destacar React, Angular, Vue.js y Svelte.

Respecto al **backend**, se puede utilizar cualquier lenguaje porque es independiente del frontend. Lo importante será crear una **API REST** que devuelva el **JSON**, de forma que aporte el contenido necesario para la Single Page Application.

Modelos de Renderizado Moderno

En la actualidad, el desarrollo web trasciende la SPA tradicional ofreciendo diversos patrones de renderizado según las necesidades del proyecto:

- CSR (Client-Side Rendering): El navegador descarga un HTML vacío con enlaces a scripts JS que construyen toda la interfaz.
- SSR (Server-Side Rendering): El servidor genera el HTML completo con datos en cada petición. Excelente para SEO y rendimiento percibido.
- SSG (Static Site Generation / Jamstack): Los archivos HTML se generan durante la fase de compilación (build time). Máxima velocidad y seguridad.
- ISR (Incremental Static Regeneration): Permite actualizar páginas estáticas en segundo plano sin recompilar todo el sitio.
- Arquitectura de Islas (Island Architecture): Renderiza la mayor parte del sitio como HTML estático e hidrata únicamente componentes interactivos (“islas”) reduciendo el peso de JavaScript.

Meta-frameworks Full-stack

Integran el frontend y backend en una misma solución gestionando el enrutamiento y los modelos de renderizado:

- Next.js (basado en React)
- Nuxt (basado en Vue.js)
- SvelteKit (basado en Svelte)
- Astro (orientado a rendimiento y arquitectura de islas)

8. Lenguajes de programación en entorno servidor y Frameworks

A continuación se muestran lenguajes y frameworks [1] para desarrollo de aplicaciones web:

Framework	Lenguaje	Características Destacadas
Express.js / NestJS	JavaScript / TypeScript	Express ofrece minimalismo; NestJS aporta una estructura modular orientada a TypeScript nativo.
Spring Boot	Java	Estándar en entornos corporativos. Excelente gestión de inyección de dependencias y seguridad.
ASP.NET Core	C# (.NET)	Framework de código abierto, multiplataforma y con un rendimiento sobresaliente.
FastAPI / Django	Python	FastAPI destaca por su velocidad asíncrona y documentación OpenAPI automática; Django por su enfoque <i>batteries-included</i> .
Laravel / Symfony	PHP	Sintaxis muy cuidada, ORM potente (Eloquent/Doctrine) y ecosistema completo.
Ruby on Rails	Ruby	Pionero en el principio de convención sobre configuración (<i>Convention over Configuration</i>).

[1] Un [framework](https://www.illusionstudio.es/que-es-un-framework-web) <<https://www.illusionstudio.es/que-es-un-framework-web>> es una plataforma para el desarrollo de aplicaciones. Puede incluir librerías y metodologías

9. Integración con los lenguajes de marcas

Existen dos estrategias para combinar la lógica de servidor con una estructura HTML:

1. Una de las formas de realizar páginas web dinámicas es **integrar el código del programa en medio de las etiquetas HTML**.

El contenido que no varía de la página se introduce en HTML, y en el lenguaje de programación todo aquello que puede variar de forma dinámica.

Cuando se solicita una página, el servidor busca en ella bloques de código. Si los encuentra, los ejecuta y los sustituye por su salida.

En PHP los bloques se delimitan por `<?php` y `?>`

```
1  <!DOCTYPE html>
2  <html>

3      <head>
4          <title>Número aleatorio</title>
5      </head>
6      <body>
7          <?php
8              echo rand(0, 100);
9          ?>
10     </body>
11 </html>
```

Al solicitar la página, el servidor ejecuta el bloque y lo sustituye por un número aleatorio entre 0 y 100. Si por ejemplo sale un 8, enviará la cliente:

```
<!DOCTYPE html>
<html>
    <head>
        <title> Número aleatorio </title>
    </head>
    <body>
        8
    </body>
</html>
```

El servidor interpreta etiquetas especiales incrustadas en el HTML antes de enviar la respuesta al navegador.

Ejemplos: Blade (Laravel), Twig (Symfony), Thymeleaf (Java), JSX/TSX en SSR (Next.js).

2. La segunda estrategia para combinar la lógica de servidor con la estructura HTML es el **desacoplamiento Frontend/Backend via API**: El backend expone datos limpios en JSON a través de una API REST/GraphQL y un framework frontend (React, Vue) renderiza los componentes de interfaz en el cliente o mediante SSR.

10. Código limpio y principios SOLID

Código limpio

El código limpio es una guía de buenas prácticas de cómo hacer buen software.

Con este conjunto de normas, elaborado por Robert C. Martin, conseguimos un software con mejor mantenimiento y alta escalabilidad.

Algunas recomendaciones:

- a. El código limpio es enfocado, el código malo hace demasiadas cosas.
- b. El lenguaje con el que se escribe el código debería parecer que fue hecho para resolver el problema
- c. El código no debe ser redundante.
- d. Debe ser placentero leer el código.
- e. Puede ser extendido fácilmente por otro desarrollador.
- f. Debe tener dependencias mínimas.
- g. Cuanto más pequeño, mejor.
- h. Debe tener pruebas unitarias y de aceptación.
- i. Debe ser expresivo.

Principios SOLID

Robert C. Martin estableció principios para facilitar a los desarrolladores la labor de crear programas legibles y mantenibles. Estos principios se llamaron SOLID (**S**ingle responsibility **O**pen/closed, **L**iskov substitution, **I**nterface segregation, **D**ependency inversión).

Son guías que pueden ser aplicadas para eliminar código sucio, provocando que el programador tenga que refactorizar el código fuente hasta que sea legible y extensible. Debe ser utilizado con el desarrollo guiado por pruebas y forma parte de la estrategia global del desarrollo ágil de software y programación adaptativa.

- a. Principio de responsabilidad única (SRP): Una clase o un método deberían tener una única responsabilidad.
- b. Principio abierto/cerrado (OCP): Todo componente debe estar abierto a nuevas funcionalidades, pero cerrado a cambios en su código
- c. Principio de sustitución de Liskov: “Si S es un subtipo de T, entonces los objetos del tipo T deberían poder ser reemplazados por objetos del tipo S sin alterar ninguna de las propiedades de la aplicación.
- d. Principio de segregación de interfaz(ISP): Las clases que implementen una interfaz o una clase abstracta no deberían estar obligadas a implementar métodos que no utilizan.
- e. Principio de inversión de dependencias(DIP): Indica que el módulo de alto nivel no debe depender del módulo de bajo nivel. Ambos deberían depender de las abstracciones. Las abstracciones no deben depender de los detalles. Son los detalles los que deberían depender de abstracciones.

Vídeo explicativo: https://www.youtube.com/watch?v=opT4C_pFgQU
<https://www.youtube.com/watch?v=opT4C_pFgQU>

11. Patrones de diseño

Los **patrones de diseño** <<https://refactoring.guru/es/design-patterns>> son la base para la búsqueda de soluciones a problemas comunes en el desarrollo de software .

Un patrón de diseño resulta ser una solución a un problema de diseño. Esta solución se habrá comprobado su efectividad para problemas similares y debe ser reutilizable en distintas circunstancias.

El uso de patrones no es obligatorio, pero sí recomendable (sin abusar de ellos o forzar su uso).

Existen varios tipos de patrones de diseño de software, entre ellos:

- Patrones de creación: Estos patrones se centran en la creación de objetos de manera flexible y eficiente. Algunos ejemplos son: Singleton, Factory Method, Abstract Factory y Builder.
- Patrones estructurales: Estos patrones se ocupan de la composición y estructura de las clases y objetos. Algunos ejemplos son: Adapter, Decorator, Composite y Proxy.
- Patrones de comportamiento: Estos patrones se centran en la interacción y comunicación entre objetos. Algunos ejemplos son: Observer, Strategy, Template Method y Command.
- Patrones arquitectónicos: Estos patrones abordan la estructura y organización de sistemas de software a gran escala. Algunos ejemplos son: Modelo-Vista-Controlador (MVC), Capas y Microservicios.
- Patrones de concurrencia: Estos patrones se utilizan para gestionar la concurrencia y la comunicación entre hilos. Algunos ejemplos son: Mutex, Semaphore, Productor-Consumidor y Monitor.

El patrón MVC

Model-View-Controller o **Modelo-Vista-Controlador** es un modelo de programación web que separa los datos y la lógica de negocio respecto a la interfaz de usuario, y el componente encargado de gestionar los eventos y las comunicaciones.

Al separar los componentes en elementos conceptuales permite reutilizar el código y mejorar su organización y mantenimiento.

Sus elementos son:

- **Modelo:** representa la información y gestiona todos los accesos a ésta, tanto consultas como actualizaciones provenientes, normalmente, de una base de datos. Se accede via el controlador.
- **Controlador:** Responde a las acciones del usuario, y realiza peticiones al modelo para solicitar información. Tras recibir la respuesta del modelo, le envía los datos a la vista.
- **Vista:** Presenta al usuario de forma visual el modelo y los datos preparados por el controlador. El usuario interactúa con la vista y realiza nuevas peticiones al controlado.



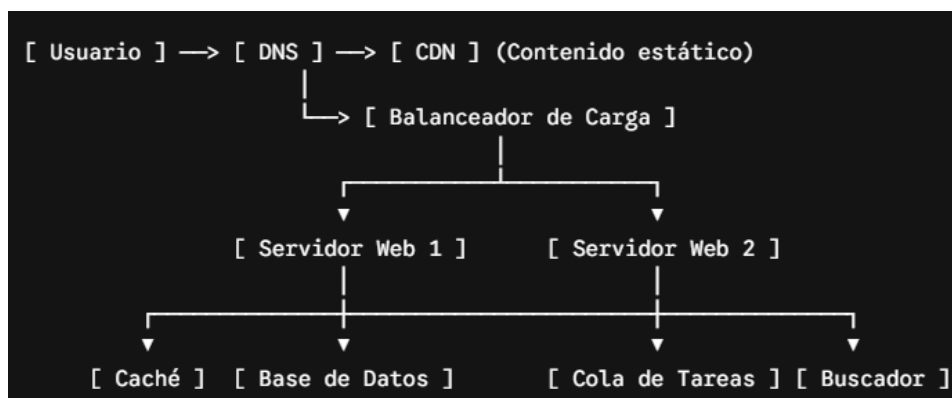
<<https://daniellamikaelson.wordpress.com/2016/07/26/patron-mvc/>>

12. Infraestructura y Despliegue

En la actualidad, las aplicaciones web de producción se ejecutan sobre infraestructuras distribuidas compuestas por múltiples servicios especializados:

Cuando usas una aplicación web (como Netflix, Instagram o Amazon), no hay un único ordenador respondiendo a todas las peticiones. Detrás de la pantalla existe todo un equipo de componentes especializados trabajando juntos para que la web cargue rápido, no se caiga aunque la usen millones de personas a la vez y guarde los datos de forma segura.

Para entenderlo fácilmente, imagina que una web moderna funciona exactamente igual que un restaurante de alta cocina:



Las piezas del sistema (analogía del restaurante):

- DNS (La dirección o el mapa): Cuando escribes miweb.com, el DNS traduce ese nombre a la dirección IP numérica exacta donde está el servidor.
- CDN (Los repartidores locales): Es una red de servidores repartidos por todo el mundo. Guarda copias de las fotos, vídeos y estilos de la web cerca de ti para servírtelos al instante sin tener que pedirlos al servidor principal. En el restaurante es como tener almacenes locales con bebidas ya frías listas para servir sin tener que ir a la fábrica central.
- Balanceador de Carga (El maître o recepcionista): Recibe todas las visitas que llegan a la web y las distribuye entre los distintos servidores disponibles para que ninguno se colapse.
- Servidores de Aplicaciones (Los cocineros / Backend): Son los ordenadores encendidos que ejecutan el código de la web (Node.js, Python, Java, etc.), procesan

tus acciones y preparan la información que vas a ver.

- Caché / ej. Redis (La mesa de ingredientes a mano): Una memoria ultra-rápida donde el servidor guarda los datos que los usuarios piden constantemente para no tener que calcularlos o buscarlos de nuevo. En el restaurante es la mesa auxiliar donde el cocinero tiene los ingredientes más usados ya picados para no perder tiempo yendo a la despensa.
- Base de Datos (La despensa principal): El lugar donde se almacena toda la información importante de forma permanente: usuarios, contraseñas, compras, publicaciones.
- Cola de Tareas / Job Queue (La lista de tareas secundarias): Se usa para procesar cosas pesadas que no hace falta mostrar de inmediato en pantalla (como enviar un email de confirmación o procesar un vídeo subido). En el restaurante es la lista de tareas como "sacar la basura" o "hacer inventario", que los ayudantes hacen en segundo plano para no frenar la preparación de los platos.

Como es una petición paso a paso:

Cuando entras a tu perfil en una web, ocurre lo siguiente en cuestión de milisegundos:

- Localización: Tu navegador le pregunta al DNS dónde está la web.
- Carga rápida: La CDN te entrega las imágenes y la estructura visual básica.
- Reparto: Tu petición para ver tus datos personales llega al Balanceador de Carga, que la deriva al servidor de aplicaciones que esté más libre.
- Consulta rápida: El servidor mira en la Caché si tus datos están guardados recientemente. Si no están, los pide a la Base de Datos.
- Respuesta: El servidor junta los datos, se los envía a tu navegador y la web se muestra completa en tu pantalla.

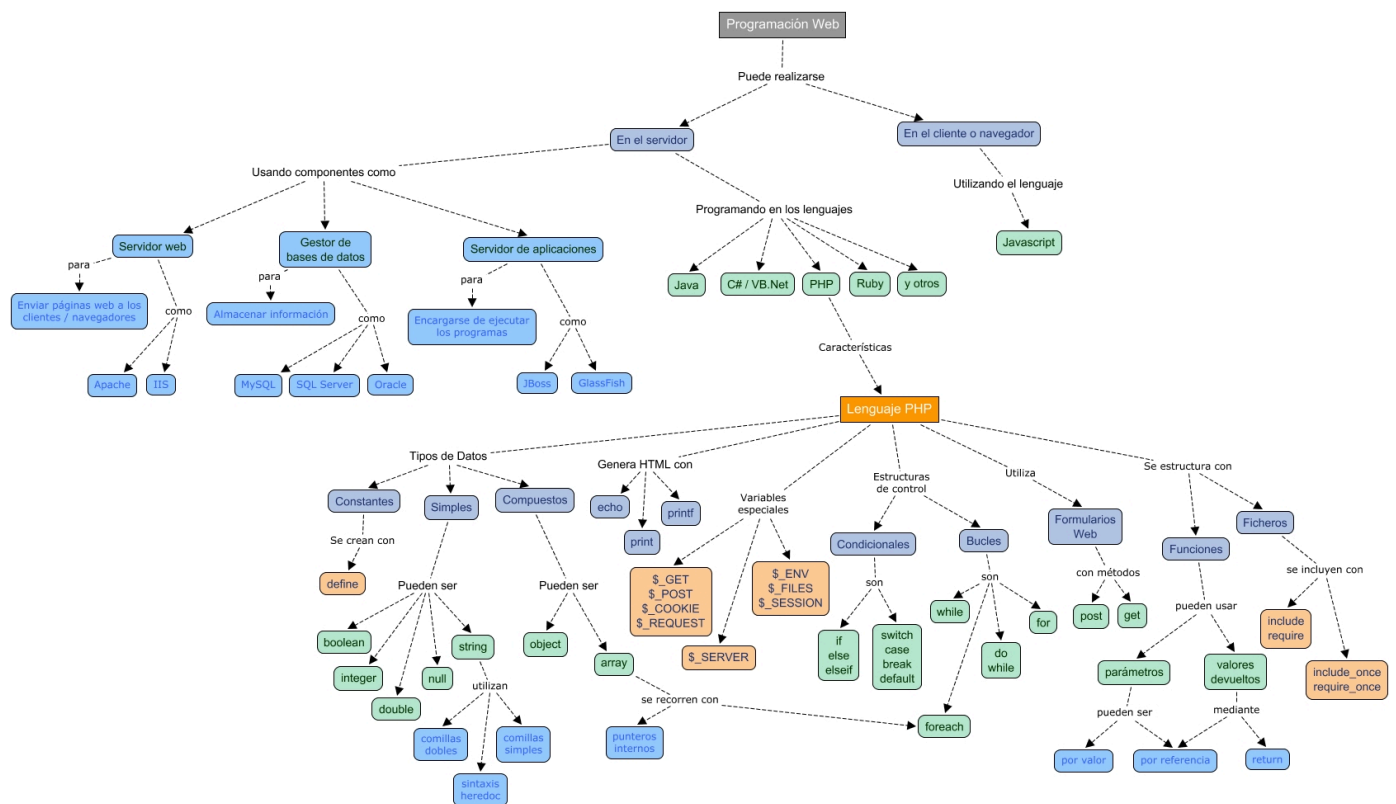
Conceptos:

Para desplegar y mantener estas webs hoy en día se utilizan tres herramientas fundamentales:

- Contenedores (Docker): Son "cajas virtuales estándar". Te permiten empaquetar tu programa con todo lo que necesita para funcionar de forma idéntica en el ordenador del programador y en los servidores de la empresa.

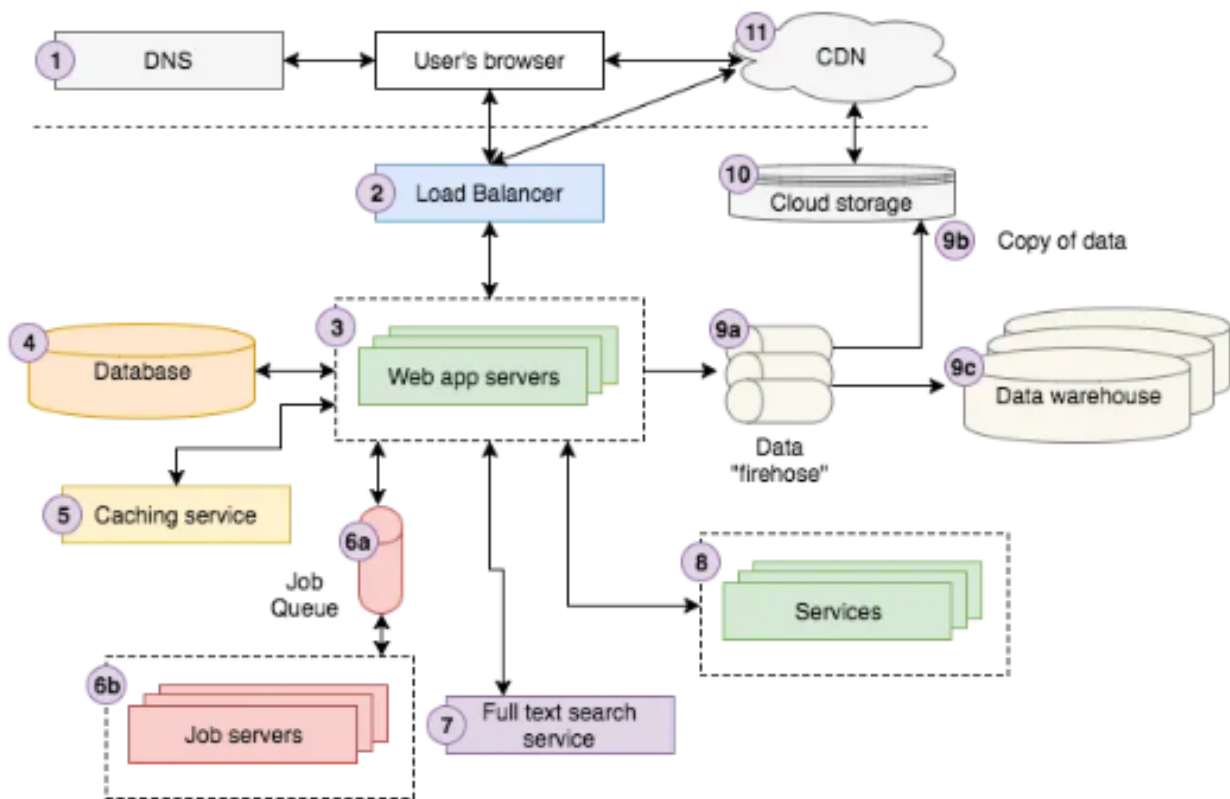
- Orquestadores (Kubernetes): Es el "director de orquesta" de los contenedores. Si entra de golpe muchísima gente a la web, Kubernetes enciende automáticamente más contenedores para absorber el tráfico y los apaga cuando la gente se va para ahorrar dinero.
- Serverless (Sin servidor tradicional): Un modelo donde no alquilas un servidor encendido las 24 horas, sino que solo pagas por los milisegundos exactos en los que se ejecuta tu código cuando un usuario realiza una acción.

13. Mapa Conceptual



Otra visión de las arquitecturas web

<<https://medium.com/@maniakhitoccori/arquitectura-web-39fe6635f6b4>> más actual:



Descripción general de la arquitectura de aplicaciones web modernas

Nota: Este documento es una recopilación de datos del libro Desarrollo web en entorno servidor de Xabier Ganzábal , el tutorial básico de JavaEE de javaHispano y de los apuntes de Desarrollo web en entorno servidor de José Luis Comesaña.



Publicada con **Licencia Creative Commons Reconocimiento Compartir igual 4.0**
<<http://creativecommons.org/licenses/by-sa/4.0/>>